- **Midterm**
  Bits Operation

same sign	unsigned rank > signed rank	signed rank > unsigned rank
`unsigned int x = 0x0000FFFF;` `unsigned short y = 0xFFFF;`	`unsigned int x = 0xFFFFFFFF;` `signed short y = 0xFFFF;`	`signed int x = 0x0000FFFF;` `unsigned short y = 0xFFFF;`

Floating Point $v = (-1)^s \times M \times 2^E$

	E(exp)	M(frac)
NaN	exp (all ones)	frac = not all zeros
$\pm\infty$	E = exp (all ones) - bias	frac = all zeros
Denorm	E = **1 - bias** (exp all zeros)	frac = M
Normalized	E = exp - bias	frac = M-1

FP mult (comm and not asso): $(-1)^{s_1} M_1 2^{E_1} \times (-1)^{s_2} M_2 2^{E_2} = (-1)^s M 2^E \quad s = s_1 \hat{} s_2 \quad M = M_1 \times M_2 \quad E = E_1 + E_2$
Assembly Caller-saved registers can be modified by any function ⇒ save the data before making calls

`mov %rax,%rsp` ≅ `rsp = rax;`	`mov (%rax),%rsp` ≅ `rsp = *(rax);`
`mov 0x1(%rax),%rsp` ≅ `rsp = *(rax+1);`	`lea 0x1(%rax),%rsp` ≅ `rsp = rax+1;`

CF(carry flag:unsigned overflow), ZF(zero flag), SF(sign flag), OF(overflow flag)

`JO`(opposed to `JNO`)	jump if overflow	OF = 1
`JS`(opposed to `JNS`)	jump if sign	SF = 1
`JE/JZ`(opposed to `JNE/JNZ`)	jump if equal / zero	ZF = 1

Stack `call`: push the return address onto the stack; `ret`: pop address from stack and jump to that address.
`push`: put the data we want to push on the stack then decrement `%rsp` by 8 bytes
≅ `subq $8, %rsp` and `movq S,%rsp`
`pop`: write the data that `%rsp` is currently pointing to into the dest and increment `%rsp` by 8 bytes
≅ `movq (%rsp), D` and `addq $8, %rsp`
Array

`int A[13]`	an array[13] of int
`int *p[13]`	an array[13] of pointer to int
`int *(p[13])`	an array[13] of pointer to int
`int (*p)[13]`	pointer to an array[13] of int
`int A[3][5]`	an int array of len 3×5
`int *A[3][5]\int *(A[3][5])`	an int pointer array of len 3×5
`int (*A)[3][5]`	pointer to an int array of len 3×5
`int (*A[3])[5]`	an array of len 3 with each pointing to an int array of len 5

Cache Capacity of a cache $C = S \times E \times B$

- **VM**
  Locality: *active pages* known as *working/resident set*; working set size exceeds the size of physical memory ⇒ *thrashing*
  Address translation:

VPN	VPO

TLBT	TLBI	VPO

PPN	PPO

tag	set index	block offset

- **Linking**
  Relocatable object files contain *ELF header*, *.text* (machine code), *.rodata* (read-only data), *.data* (init glbal static vars), *.bss* (uninitialized global and static vars + any that are init to zero), *.symtab* (symbol table), *.rel.text* (locations that will be modified), *.rel.data* (globals referenced/defined externally), *.debug* (debugging symbol table), *.line* (mapping between line no. of source program and machine code instructions), *.strtab* (string table)

Symbol resolution:
functions and initialized globals are strong symbols; uninit globals are weak
1) multiple strong symbols with the same name are not allowed
2) given a strong symbol and multiple weak symbols with the same name, choose the strong symbol
3) given multiple weak symbols with the same name, choose any of the weak symbols

- **I/O**

`int open(char *filename, int flags, mode_t mode);` returns new file descriptor if OK, -1 on error
`flags`: `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREAT` (if the file doesn't exists, create an empty version), `O_TRUNC` (if the file exists, truncate it), `O_APPEND` (before each write, set the file position to the end)
`mode`: `S_IRUSR`, `S_IWUSR`, `S_IXUSR` `S_IRGRP`, `S_IWGRP`, `S_IXGRP` `S_IROTH`, `S_IWOTH`, `S_IXOTH`

`int close(int fd);` returns 0 if OK, -1 on error
`ssize_t read(int fd, void *buf, size_t n);` returns number of bytes read if OK, 0 on EOF, -1 on error
`buf` **stays unchanged if EOF is reached**
`ssize_t write(int fd, void *buf, size_t n)` returns number of bytes written if OK, 0 on EOF, -1 on error

`ssize_t rio_readn(int fd, void *usrbuf, size_t n);`
`ssize_t rio_writen(int fd, void *usrbuf, size_t n);` returns number of bytes transferred if OK, 0 on EOF, -1 on error
`ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen);`
`ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n);` returns numbers of bytes read if OK, 0 on EOF, -1 on error

Sharing: Each process has its own separate *descriptor table*; the set of open file is represented by a *file table* that is shared by all processes; the *v-node table* is shared by all processes
The child gets its own copy of the parent's descriptor table. Parent and child share the same set of open file tables and must share the same file position.

I/O redirection: `int dup2(int oldfd, int newfd);` returns nonnegative descriptor if OK, -1 on error
copies descriptor table entry `oldfd` to descriptor table entry `newfd` by overwriting

- **Processes**

Each program is provided with its own private address space which in general cannot be read or written by any other process
A context switch can occur while the kernel is executing a system call that may block on behalf of the user as as result of interrupt

`getpid` returns the PID of the calling process
`getppid` returns the PID of its parent
`pid_t getpgrp();` returns process group ID of the calling process
By default, a child process belongs to the same process group as its parent
`setpgid(pid_t pid, pid_t pgid);` returns 0 on success, -1 on error
If `pid` is 0, the PID of the prcess is used; if `pgid` is 0, the PID of the process specified by `pid` is used for the process group ID

`fork()`: the PID of the child process is returned in the parent, and 0 is returned in the child
Since the parent and the child are separate processes, they each have their own private address spaces. Any subsequent changes that a parent or child makes are not reflected in the memory of other processes.
The child inherits all of the parent's open files.
`pid_t waitpid(pid_t pid, int *statusp, int options);` returns PID of child if OK, 0 if `WNOHANG`, -1 on error (no child)
By default, it suspends execution of the calling process until a child process in its wait set terminates. The wait set is

$$\begin{cases} \text{the singleton child process whose PID} = \texttt{pid} & \text{if } \texttt{pid} > 0 \\ \text{all of the parent's child processes} & \text{if } \texttt{pid = -1} \end{cases}$$

`WEXITSTATUS` returns the exit status of the child, defined only if `WIFEXITED()`. This consists of the least significant 8 bits of the status argument that the child specified in a call to `exit()` or `_exit()` oras the argument for a `return` statement in `main()`;
`WTERMSIG` returns the signal number that caused the child process to terminate, defined only if `WIFSIGNALED()`;
`WSTOPSIG` returns the signal number that caused the child to stop, defined only if `WIFSTOPPED()`;
`WNOHANG` return immediately 0 if none of the child has terminated yet
`WUNTRACED` check both terminated and stopped children
`WCONTINUED` suspend execution until a process in the wait set is terminated or a stopped process has been resumed by a `SIGCONT`

`pid_t wait(int *statusp);` returns PID of child if OK or -1 on error
`wait(&status)` ≅ `waitpid(-1, &status, 0)`

`unsigned int sleep(unsigned int secs);` returns seconds left to sleep
`int pause();` returns -1, puts the calling function to sleep until a signal is received by the process

`int execve(const char *filename, const char *argv[], const char *envp[]);` doesn't return if OK, -1 on error

- **Signals**

int kill(pid_t pid, int sig) returns 0 if OK, -1 on error
`kill` returns after the signal is sent
signal sent to self must be handled before kill returns; signal sent to others takes time to arrive
sometimes signals may not arrive before the target returns

if `pid = 0`, then `kill` sends `sig` to every process in the process group of the calling process (including itself)
`<` `|pid|`

`Default action`: terminates; terminates and dumps core; stops until restarted by `SIGCONT`; ignores
`SIGKILL` **and** `SIGSTOP` **cannot be caught, blocked, or ignored**

Implicit blocking: by default, the kernel blocks any pending signals of the type currently being processed by a handler
Explicit blocking:
`int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);` `int sigemptyset(sigset_t *set);`
`int sigfillset(sigset_t *set);` (`int sigaddset(sigset_t *set, int signum);`)
`int sigdelset(sigset_t *set, int signum);` returns 0 if OK, -1 on error
`int sigismember(cosnt sigset_t *set, int signum);` returns 1 if member, 0 if not, -1 on error
`SIG_BLOCK`: add the signals in `set` to `blocked`
`SIG_UNBLOCK`: remove the signals in `set` from `blocked`
`SIG_SETMASK`: `blocked = set`

Signals cannot be used to count the occurrence of events in other processes
Explicit waiting for signals: `int sigsuspend(const sigset_t *mask);`
returns -1, temporarily replaces the current blocked set with `mask` then suspend the process until the receipt of a signal

Resource	Threads don't share	Processes don't share
Stack pointer	✓	✓
Code	×	✓
Date	×	✓
Signal mask	✓	✓

- **Threads / Threads Sync**

Each Internet host has the locally defined domain name `localhost`, which always maps to the *loopback address* `127.0.0.1`

`int socket(int domain, int type, int protocol);`

returns nonnegative descriptor if OK, -1 on error, creates a *socket descriptor*

`int connect(int clientfd, const struct sockaddr *addr, socklen_t addrlen);`

returns 0 if OK, -1 on error, establishes an Internet connection with the server at sock address `addr`, blocks until the connection is successfully established or an error occurs

`int bind(int sockfd, const struct sockaddr *addr, coklen_t addrlen);`

returns 0 if OK, -1 on error, asks the kernel to associate the server's socket address in `addr` with the socket descriptor

`int listen(int sockfd, int backlog);` returns 0 if OK, -1 on error

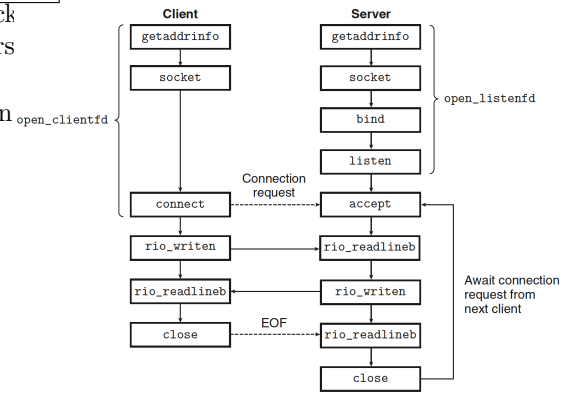
converts `sockfd` from an active socket to a *listening socket*

`int accept(int listenfd, struct sockaddr *addr, int *addrlen);`

returns nonnegative connected descriptor if OK, -1 on error

`int open_clientfd(char *hostname, char *port);`

`int open_listenfd(char *port);` returns descriptor if OK, -1 on error



`int pthread_create(pthread_t *tid, pthread_attr_t *attr, func *f, void *arg);` returns 0 if OK, nonzero on error

`pthread_t pthread_self();` returns thread ID of caller

`void pthread_exit(void *thread_return);` never returns

If the main thread calls `pthread_exit`, it waits for all other peer threads to terminate and then terminates the main thread and the entire process with a return value of `thread_return`

int pthread_cancel(pthread_t tid); returns 0 if OK, nonzero on error

`int pthread_join(pthread_t tid, void **thread_return);` returns 0 if OK, nonzero on error

It cannot wait for an arbitrary thread to terminate

A thread is *joinable* or *detached*

`int pthread_detach(pthread_t tid);` returns 0 if OK, nonzero on error

`int pthread_once(pthread_once_t *once_control, void (*init_rountine)(void));` initializes the state associated with a thread routine

1) A global variable is any variable declared outside of a function. At run time, the read/write area of virtual memory contains exactly one instance of each global variable that can be referenced by any thread

2) A local automatic variable is one that is declared inside a function without the `static` attribute. At run time, each thread's stack contains its own instances of any local automatic variables

3) A local static variable is one that is declared inside a function with the `static` attribute . The read/write area of virtual memory contains exactly one instance of each local static variable declared in a program

Thread-unsafe: functions that don't protect shared variables; keep state across multiple invocations; return a pointer to a static variable; call thread-unsafe functions

Reentrant functions don't reference any shared data when they're called by multiple threads

A race occurs when the correctness of a program depends on one thread reaching point x in its control flow before another thread reaches point y

A deadlock is when a collection of threads is blocked, waiting for a condition that will never be true

Mutual lock ordering rule: Given a total ordering of all mutexes, a program is deadlock-free if each thread acquires its mutexes in order and release them in reverse order

- **Miscellaneous**

  In GDB, `x {address}` is used to examine a memory location

  `int x = 0; printf("%d %d", x++, x);` ⇒ `0, 1`

Bit Operation

1. C promotion rules

   - (Text book): When an operation is performed where one operand is signed and the other is unsigned, C implicitly casts the signed argument ot unsigned and performs the operations
   - If both operands are of signed or unsigned type, the operand with the type of less integer conversion rank is converted to the type of the operand with greater rank
   - Otherwise, if the unsigned operand has greater rank, the signed operand with less rank is promoted to the unsigned type with greater rank with sign extension (prserving value is priority)
   - Otherwise, the signed operand has a greater rank and can represent all possible value of the unsigned type of the other operand, then the unsigned operand is promoted to the signed type with greater rank with value unchanged

same sign	unsigned rank > signed rank	signed rank > unsigned rank
`unsigned int x = 0x0000FFFF;` `unsigned short y = 0xFFFF;`	`unsigned int x = 0xFFFFFFFF;` `signed short y = 0xFFFF;`	`signed int x = 0x0000FFFF;` `unsigned short y = 0xFFFF;`

2. conversion order: first change the size then the type

3. Integer division always rounds toward zero: $\begin{cases} \lfloor x/y \rfloor & x \geq 0, y > 0 \text{ or } x < 0, y < 0 \\ \lceil x/y \rceil & x < 0, y > 0 \text{ or } x \geq 0, y < 0 \end{cases}$
   Two's-complement division by a power of 2, rounding down: $\lfloor x/2^k \rfloor$
   rounding up: $(x + (1 << k) - 1) >> k = \lceil x/2^k \rceil$

4. (Midterm mistake): C treats nonzero as true:
   `(((!!x) << 31) >> 31) & x == x` is true
   `x >= 0` $\Rightarrow$ `((!x - 1) & x) == x` is true

Floating Point

1. $v = (-1)^s \times M \times 2^E$

2. bias $= 2^{k-1} - 1, k = \#$ bits in exp field

   - single precision: bias 127 (k=8), $exp \in [1, 254], E \in [-126, 127]$
   - double precision: bias 1023 (k=11), $exp \in [1, 2046], E \in [-1022, 1023]$

	E(exp)	M(frac)
NaN	exp (all ones)	frac = not all zeros
$\pm\infty$	E = exp (all ones) - bias	frac = all zeros
Denorm	E = **1 - bias** (exp all zeros)	frac = M
Normalized	E = exp - bias	frac = M-1

3. (float to bit): write float in binary $\Rightarrow$ adjust the binary point to get proper $M, E \Rightarrow$ calculate exp and frac according to the table above

4. Round-to-even rounds the halfway number (x.x...x‖100...) such that the least significant bit of the result is even

5. FP mult (commutative and not associative): $(-1)^{s_1} M_1 2^{E_1} \times (-1)^{s_2} M_2 2^{E_2} = (-1)^s M 2^E$
   $s = s_1 \hat{} s_2 \qquad M = M_1 \times M_2 \qquad E = E_1 + E_2$

Assembly

1. char: byte(b); short: word(w)/2 byte; int: double word(l); long: quad word(q); pointer: quad word
2.

64	32	16	
%rax	%eax	%ah	%al

3. First 6 arguments in register %rdi, %rsi, %rdx, %rcx, %r8, %r9, the rest are on the stack
4. Values of callee-saved registers (%rbx, %rbp, %r12-%r15) must be preserved by the procedure called
   Caller-saved registers can be modified by any function ⇒ save the data before making calls
5. Instructions generating 4-byte quantities set the upper 4 bytes of register to 0 (ex. `movl`)
6. `MOV`: `movz` zero extension and `movs` sign extension
7. `mov %rax,%rsp` ≅ `rsp = rax;`
   `mov (%rax),%rsp` ≅ `rsp = *(rax);`
   `mov 0x1(%rax),%rsp` ≅ `rsp = *(rax+1);`
   `lea 0x1(%rax),%rsp` ≅ `rsp = rax+1;`
8. CF(carry flag:unsigned overflow), ZF(zero flag), SF(sign flag), OF(overflow flag)

`JO`(opposed to `JNO`)	jump if overflow	OF = 1
`JS`(opposed to `JNS`)	jump if sign	SF = 1
`JE/JZ`(opposed to `JNE/JNZ`)	jump if equal / zero	ZF = 1

Stack

1. `call`: push the return address onto the stack
2. `ret`: pop address from stack and jump to that address
3. `push`: decrement `%rsp` by 8 bytes then put the data we want to push on the stack
   ≅ `subq $8, %rsp` and `movq S,%rsp`
4. `pop`: write the data that `%rsp` is currently pointing to into the dest and increment `%rsp` by 8 bytes
   ≅ `movq (%rsp), D` and `addq $8, %rsp`
   (Midterm mistake): `pop` accesses memory

Array

1. size of `struct` = $\sum$ size of all fields; size of `union` = max{field sizes}
2.

`int A[13]`	an array[13] of int
`int *p[13]`	an array[13] of pointer to int
`int *(p[13])`	an array[13] of pointer to int
`int (*p)[13]`	pointer to an array[13] of int
`int A[3][5]`	an int array of len 3×5
`int *A[3][5]\int *(A[3][5])`	an int pointer array of len 3×5
`int (*A)[3][5]`	pointer to an int array of len 3×5
`int (*A[3])[5]`	an array of len 3 with each pointing to an int array of len 5

3. string in C: final character = `\0`

Caches

1. Capacity of a cache $C = S \times E \times B$
   $B = 2^b$ bytes per cache block, b is # bits of block offset; $S = 2^s$ sets, s is # set bits; E is associativity

2. Address: $t + s + b = m$, total number of bits in an address

(tag) t bits	(set index) s bits	(block offset) b bits